COLLEGE OF COMPUTING AND INFORMATICS

PUTRAJAYA CAMPUS

SEMESTER 2 2025/2026

LAB 3 & 4: COURSE PROFILE

SUBJECT CODE: CSEB5223

SUBJECT: SOFTWARE CONSTRUCTION AND METHODS

PREPARED BY: Group 3 (Section 01BT)

NAME OF MEMBERS:

No.	Full Name	Student ID
1.	Wan Jehan Farouq bin Wan Ahmed	BSW01084722
2.	Thaqif bin Nabil	BSW01084719
3.	Muhammad Haikal Sharafuddin bin Shahrol Azam	BSW01084695
4.	Muhammad Danesh Haikimi Lou	BSW01084693

Table of Contents

Introduction

The UniLearn system is developed primarily using HTML to structure and present the user interface of the application. To comply with the requirement of applying object-oriented programming (OOP) practices, JavaScript is integrated into the system to handle logic and functionality. Through JavaScript, key features are implemented using OOP concepts such as classes, objects, and modular design, ensuring the system is well-structured, reusable, and maintainable while still fulfilling the functional requirements.

1.0 Screenshots

1.1 Add Course

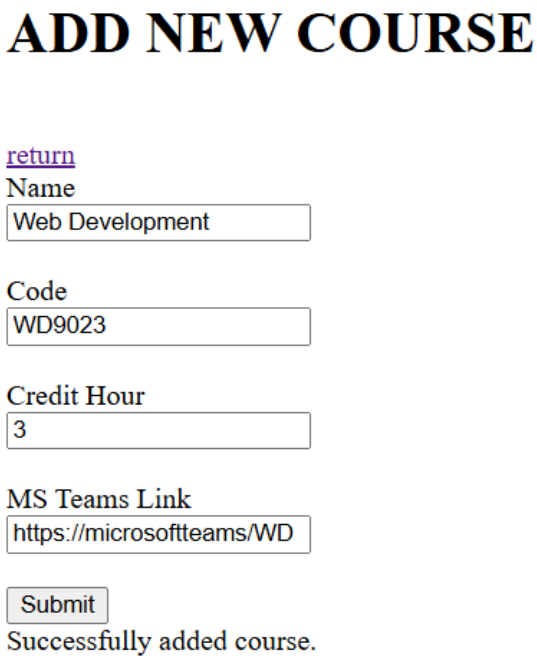

Figure 1: Section to add new course

The screenshot shows the "Add New Course" section of the UniLearn system, where users can create a new course. The interface includes input fields such as course name, course code, credit hours and Microsoft Teams Link along with a submission button to save the course details. A simple successful message will be displayed upon submission.

1.2 Search Course

Figure 2: Section to search for courses

The screenshot shows the "Search Course" section of the UniLearn system, where users can find existing courses quickly. The interface includes a search bar that allows users to enter keyword such as course code, and press "Enter" to display relevant results.

1.3 Edit Course

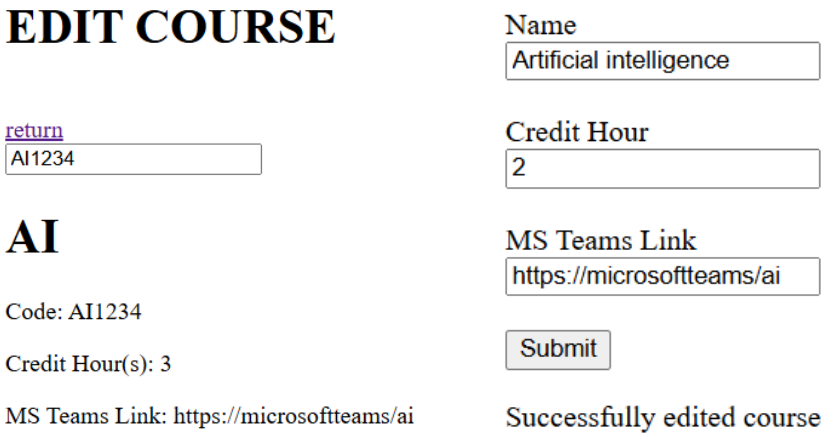

Figure 3: Section to edit course details

The screenshot shows the "Edit Course" section of the UniLearn system, where users can update existing course information. First, users need to enter the course code that they want to edit. Then, the interface will display editable fields such as course name, credit hours and MS Teams Link along with a submit button to save the changes. A simple successful message will be displayed upon submission.

1.4 Delete Course

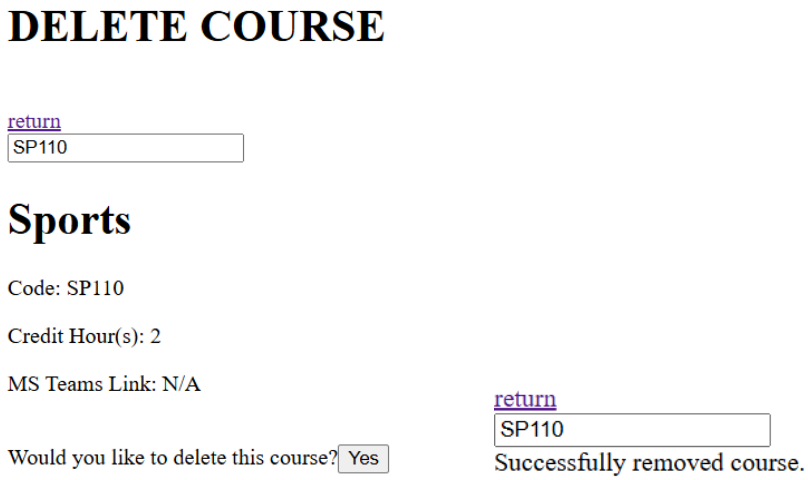

Figure 4: Section to delete courses

The screenshot shows the "Delete Course" section of the UniLearn system, where users can remove an existing course from the system. The system will allow users to search for a specific course by entering the course code, along with a confirmation prompt to confirm the action. A simple successful message will be displayed upon deletion.

1.5 View all Courses

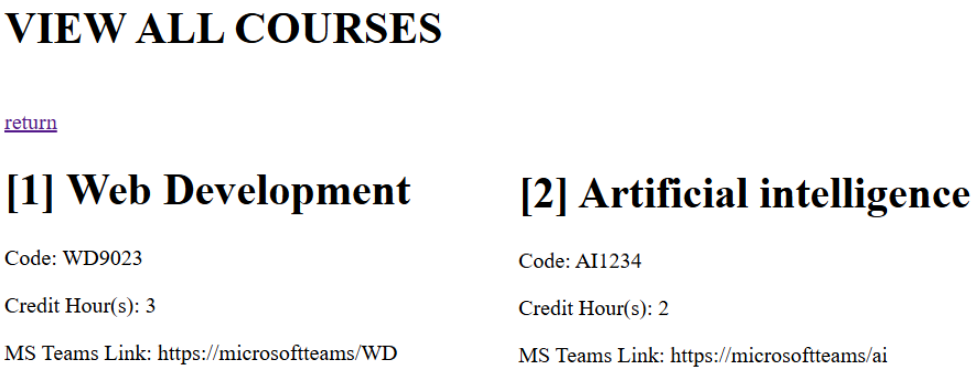

Figure 5: Section to view all courses

The screenshot shows the "View All Courses" section of the UniLearn system, where users can see a complete list of all available courses. The interface will display course details such as course name, course code, credit hours and MS Teams Link in a structured format. This section allows users to easily browse and review all course information within the system.

2.0 Analysis of Software Construction Best Practices and Challenges

2.1 Best Practices

i) Implementation of Core Functionality

The UniLearn system successfully supports essential operations such as adding, viewing, searching, editing, and deleting course profiles. For example, users can input a new course in the "Add New Course" section and immediately see it listed in the "View All Courses" section, showing that the system processes and displays data correctly.

ii) Use of Object-Oriented Programming (OOP)

JavaScript is used to implement OOP concepts such as classes and objects. For instance, a Course class is used to represent course data (course name, course code, credit hours, summary, ms teams link), this improves code organization and reusability:

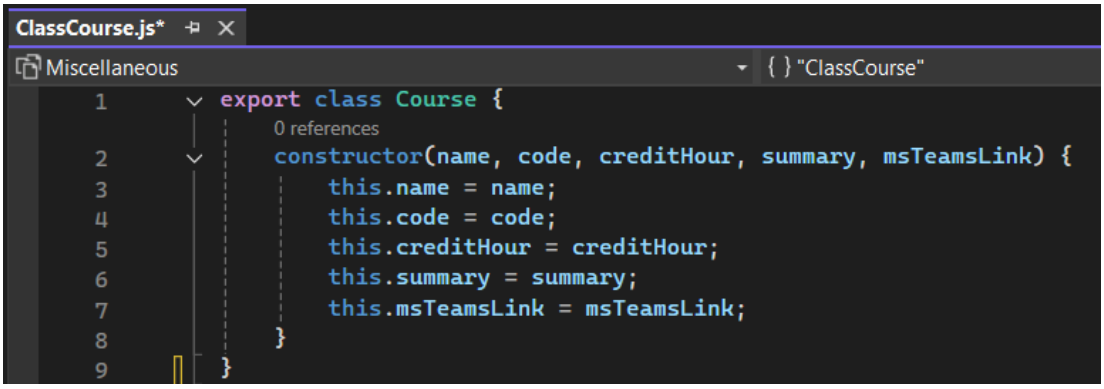

Figure 6: Course class (javascript)

iii) Consistent Documentation Standards

Each file includes header comments describing its purpose and functions. This helps developers understand how each part of the system works without confusion:

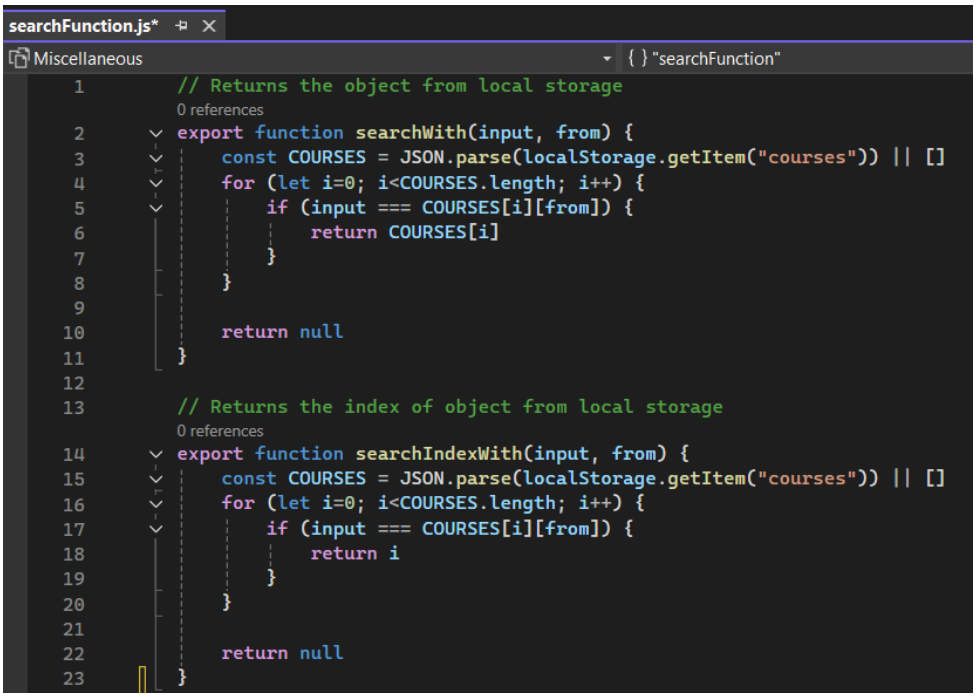

Figure 7: Code Commenting

iv) Clean and Organized Code Structure

The code follows consistent naming conventions such as camelCase for variables (e.g., creditHour) and PascalCase for classes (e.g., Course). Files are also organized logically, separating HTML structure and JavaScript logic, making the system easier to maintain.

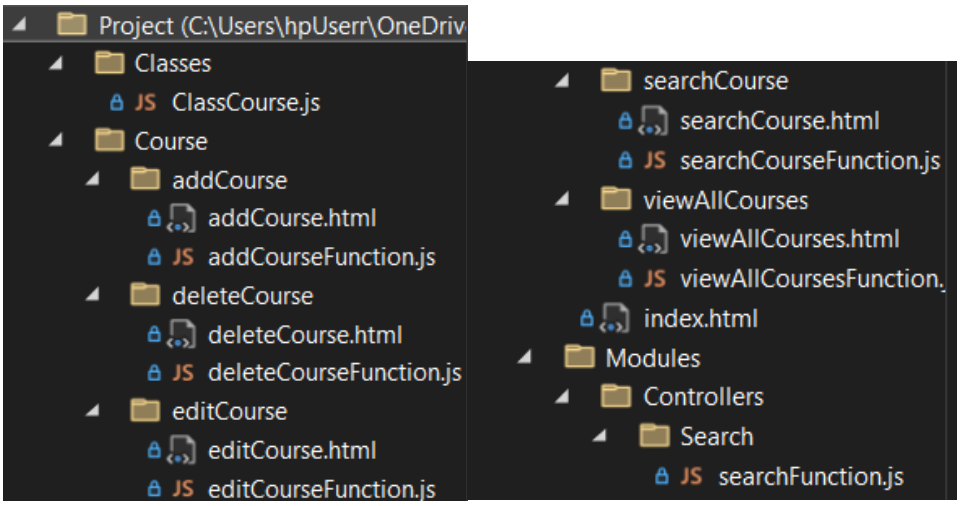

Figure 8: File Organization

v) User-Friendly Interface Design

The interface is simple and intuitive. For example, in the “Search Course” section, users only need to enter the course code in a search bar and press “Enter” to retrieve results, making the system easy to use even for beginners.

vi) Basic Error Handling Implementation

The system includes input validation to prevent errors. For example, when searching a course, the system checks if required fields like course name and course code are enter properly before providing results, reducing the risk of invalid data.

2.1 Challenges

i) Integration Between HTML and JavaScript

Handling dynamic updates between HTML and JavaScript can be challenging. For example, after deleting a course, ensuring that the “View All Courses” list updates immediately without refreshing the page requires careful coding.

ii) Maintaining OOP Structure in JavaScript

Applying OOP concepts consistently can be difficult. For instance, ensuring that all course-related operations (add, edit, delete) are properly encapsulated within the Course class or related modules requires proper design planning.

iii) Ensuring Consistent Coding Standards

When multiple features are developed (e.g., Add Course, Search Course, Edit Course), maintaining consistent naming and formatting across all JavaScript files can be challenging, especially in group work.